Projet JEE – MVC1

(Servlet, JSP & JSTL)

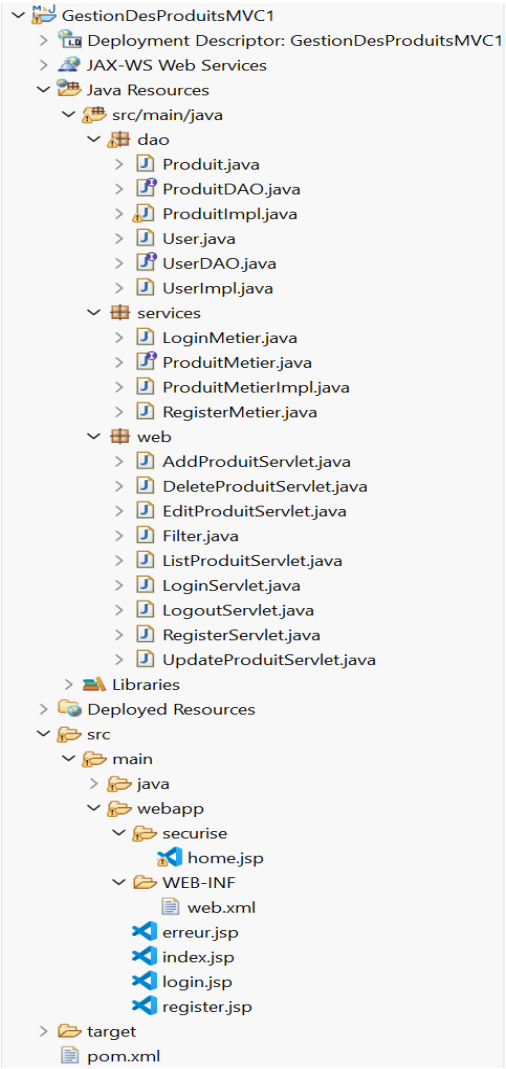
A.U : 2025 / 2026

Module : Applications web avancées avec Java et Spring

Cycle ingénieur : TDIA2

Nom Complet : Ilyas BEL EL YAZID

✓ Architecture du projet



✓ Implémentation du projet

1. Descripteur du deployement (web.xml)

```xml
<?xml version="1.0" encoding="UTF-8"?>
<web-app
    version="4.0"
    xmlns="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:javaee="http://xmlns.jcp.org/xml/ns/javaee"
    xmlns:xml="http://www.w3.org/XML/1998/namespace"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd">

    <!-- Nom affiché de l'application web -->
    <display-name>GestionDesProduitsMVC1</display-name>

    <!-- Liste des fichiers d'accueil (welcome files) -->
    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
        <welcome-file>index.htm</welcome-file>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>

    <servlet>
        <servlet-name>login</servlet-name>
        <servlet-class>web.LoginServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>login</servlet-name>
        <url-pattern>/LoginServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>logout</servlet-name>
        <servlet-class>web.LogoutServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>logout</servlet-name>
        <url-pattern>/LogoutServlet</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>register</servlet-name>
        <servlet-class>web.RegisterServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>register</servlet-name>
        <url-pattern>/RegisterServlet</url-pattern>
    </servlet-mapping>

    <!-- Déclaration de la servlet qui liste les produits -->
    <servlet>
        <servlet-name>list</servlet-name> <!-- nom logique de la servlet -->
        <servlet-class>web.ListProduitServlet</servlet-class> <!-- classe Java correspondante -->
    </servlet>
    <servlet-mapping>
        <servlet-name>list</servlet-name>
        <url-pattern>/listProduits</url-pattern> <!-- URL pour accéder à cette servlet -->
    </servlet-mapping>
```

```xml
    <!-- Déclaration de la servlet qui ajoute un produit -->
    <servlet>
        <servlet-name>add</servlet-name>
        <servlet-class>web.AddProduitServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>add</servlet-name>
        <url-pattern>/addProduit</url-pattern>
    </servlet-mapping>

    <!-- Déclaration de la servlet qui supprime un produit -->
    <servlet>
        <servlet-name>delete</servlet-name>
        <servlet-class>web.DeleteProduitServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>delete</servlet-name>
        <url-pattern>/deleteProduit</url-pattern>
    </servlet-mapping>

    <!-- Déclaration de la servlet qui charge un produit pour modification -->
    <servlet>
        <servlet-name>edit</servlet-name>
        <servlet-class>web.EditProduitServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>edit</servlet-name>
        <url-pattern>/editProduit</url-pattern>
    </servlet-mapping>

    <!-- Déclaration de la servlet qui met à jour un produit existant -->
    <servlet>
        <servlet-name>update</servlet-name>
        <servlet-class>web.UpdateProduitServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>update</servlet-name>
        <url-pattern>/updateProduit</url-pattern>
    </servlet-mapping>

    <!-- GESTION GLOBALE DES ERREURS -->
    <!-- Intercepter l'erreur 404 (Page non trouvée) -->
    <error-page>
        <error-code>404</error-code>
        <location>/erreur.jsp</location>
    </error-page>

    <!-- Intercepter l'erreur 500 (Erreur interne du serveur / Code Java qui plante) -->
    <error-page>
        <error-code>500</error-code>
        <location>/erreur.jsp</location>
    </error-page>

    <!-- Intercepter TOUTES les exceptions Java non gérées (ex: NullPointerException) -->
    <error-page>
        <exception-type>java.lang.Throwable</exception-type>
        <location>/erreur.jsp</location>
    </error-page>

</web-app>
```

2. Couche entities

Produit.java

```java
package dao;

/**
 * Classe représentant un produit dans l'application. Cette classe sert de
 * modèle pour le CRUD. Chaque instance correspond à un produit unique avec ses
 * propriétés.
 */

public class Produit {
    private long idProduit; // Identifiant unique du produit (généralement auto-incrémenté en BDD)
    private String nom; // Nom du produit
    private String description; // Description détaillée du produit
    private double prix; // Prix du produit

    // Constructeur vide requis par Spring et JSP pour créer des objets
    // dynamiquement
    public Produit() {
        super();
    }

    // Constructeur utile pour créer rapidement un produit sans ID (ex: lors de
    // l'ajout)
    public Produit(String nom, String description, double prix) {
        super();
        this.nom = nom;
        this.description = description;
        this.prix = prix;
    }

    // --- Getters et Setters ---
    // Ils permettent à Spring MVC et aux JSP de lier automatiquement les valeurs

    public long getIdProduit() {
        return idProduit;
    }

    public void setIdProduit(long idProduit) {
        this.idProduit = idProduit;
    }

    public String getNom() {
        return nom;
    }

    public void setNom(String nom) {
        this.nom = nom;
    }

    public String getDescription() {
        return description;
    }

    public void setDescription(String description) {
        this.description = description;
    }

    public double getPrix() {
        return prix;
    }

    public void setPrix(double prix) {
        this.prix = prix;
    }
}
```

User.java

```java
package dao;

public class User {
    private String firstname;
    private String lastname;
    private String email;
    private String password;
    private String role;

    public User() {
        super();
    }

    public User(String firstname, String lastname, String email, String password, String role) {
        super();
        this.firstname = firstname;
        this.lastname = lastname;
        this.email = email;
        this.password = password;
        this.role = role;
    }

    public String getFirstname() {
        return firstname;
    }

    public void setFirstname(String firstname) {
        this.firstname = firstname;
    }

    public String getLastname() {
        return lastname;
    }

    public void setLastname(String lastname) {
        this.lastname = lastname;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPassword() {
        return password;
    }

    public void setPassword(String password) {
        this.password = password;
    }

    public String getRole() {
        return role;
    }
    }

    public void setRole(String role) {
        this.role = role;
    }
}
```

3. Couche DAO

ProduitDAO.java

```java
package dao;

import java.util.List;

/**
 * Interface DAO pour la gestion des produits. Définit les opérations CRUD sans
 * préciser la méthode de persistance. L'implémentation concrète peut être en
 * mémoire, JDBC, JPA, etc.
 */

public interface ProduitDAO {

    /**
     * Ajouter un nouveau produit
     *
     * @param p Le produit à ajouter
     */
    public void addProduit(Produit p);

    /**
     * Supprimer un produit par son identifiant
     *
     * @param id L'identifiant du produit à supprimer
     */
    public void deleteProduit(long id);

    /**
     * Récupérer un produit à partir de son identifiant
     *
     * @param id L'identifiant du produit
     * @return Le produit correspondant ou null si non trouvé
     */
    public Produit getProduitById(long id);

    /**
     * Récupérer la liste de tous les produits
     *
     * @return Liste complète des produits
     */
    public List<Produit> getAllProduits();

    /**
     * Mettre à jour un produit existant
     *
     * @param p Produit avec les nouvelles informations (l'id doit être renseigné)
     */
    public void updateProduit(Produit p);

}
```

ProduitImpl.java

```java
package dao;

import java.util.ArrayList;

/**
 *  * Implémentation concrète de ProduitDAO utilisant une liste en mémoire.  *
 * Utile pour tester l'application sans base de données.  * Chaque produit
 * ajouté se voit attribuer un identifiant unique automatiquement.
 */

public class ProduitImpl implements ProduitDAO {
    private List<Produit> produits = new ArrayList<Produit>(); // Liste simulant la BDD

    /**
     * Initialisation de quelques produits pour test.
     * Méthode appelée par Tomcat au démarrage.
     */
    public void init() {
        System.out.println("Spring IOC est bien fonctionnée !");

        addProduit(new Produit("PC 1","Sony vaio 1",7000.0));
        addProduit(new Produit("PC 2","Sony vaio 2",6000.0));
    }

    /**
     * Ajouter un produit à la liste.
     * @param p Le produit à ajouter
     * L'ID est généré automatiquement (size + 1)
     */
    @Override
    public void addProduit(Produit p) {
        p.setIdProduit(new Long(produits.size() + 1));
        produits.add(p);
    }

    /**
     * Supprimer un produit par son ID
     * @param id Identifiant du produit à supprimer
     */
    @Override
    public void deleteProduit(long id) {
        produits.remove(getProduitById(id));
        // Variante commentée: suppression via boucle explicite
        /*
        for(Produit p:produits){
            if(p.getIdProduit().equals(id)){
                produits.remove(p);
                break;
            }
        }
        */
    }

    /**
     * Récupérer un produit via son ID
     * @param id Identifiant du produit
     * @return Produit correspondant ou null si non trouvé
     */
    @Override
    public Produit getProduitById(long id) {
        Produit produit = null;
        for(Produit p : produits){
            if(p.getIdProduit() == id){
                produit = p;
                break; // on arrête la boucle dès qu'on trouve le produit
            }
        }
        return produit;
    }

    /**
     * Récupérer tous les produits
     * @return Liste complète des produits
     */
    @Override
    public List<Produit> getAllProduits() {
        return produits;
    }

    /**
     * Mettre à jour un produit existant
     * @param p Produit contenant les nouvelles informations
     */
    @Override
    public void updateProduit(Produit p) {
        for (int i = 0; i < produits.size(); i++) {
            Produit existingProduit = produits.get(i);
            if (existingProduit.getIdProduit() == p.getIdProduit()) {
                // Mise à jour des champs
                existingProduit.setNom(p.getNom());
                existingProduit.setDescription(p.getDescription());
                existingProduit.setPrix(p.getPrix());
                break; // on sort après mise à jour
            }
        }

    }

}
```

UserDAO.java

```java
package dao;

public interface UserDAO {
    User getUser(String email, String password);

    User getUser(String email);

    void ajouterUser(User user);

    void supprimerUser(User user);

    void modifierUser(User user);
}
```

UserImpl.java

```java
package dao;

import java.util.ArrayList;

public class UserImpl implements UserDAO {

    public static List<User> bd = new ArrayList<>();

    /**
     * Initialisation de quelques produits pour test.
     * Méthode appelée par Tomcat au démarrage.
     */
    public void init() {
        ajouterUser(new User("ilyas","belelyazid","ilyasbelelyazid@gmail.com","123", "admin"));
        ajouterUser(new User("abc","abc","abc@gmail.com","ensah", "client"));
    }

    @Override
    public void ajouterUser(User user) {
        bd.add(user);
    }

    @Override
    public User getUser(String email, String password) {
        for (User u : bd)
            if (u.getEmail().equals(email) && u.getPassword().equals(password))
                return u;
        return null;
    }

    @Override
    public User getUser(String email) {
        for (User u : bd)
            if (u.getEmail().equals(email))
                return u;
        return null;
    }

    @Override
    public void supprimerUser(User user) {
        // TODO Auto-generated method stub
    }

    @Override
    public void modifierUser(User user) {
        // TODO Auto-generated method stub
    }
}
```

4. Couche métiers (services)

ProduitMetier.java

```java
package services;

import java.util.List;

/**
 * Interface du service métier pour la gestion des produits.
 * Cette couche sert d'intermédiaire entre le DAO (accès aux données)
 * et le Controller (gestion des requêtes HTTP).
 *
 * Elle peut contenir de la logique métier supplémentaire (validation, calculs, règles, etc.)
 */

public interface ProduitMetier {

    /**
     * Ajouter un nouveau produit
     * @param p Produit à ajouter
     */
    public void addProduit(Produit p);

    /**
     * Supprimer un produit par son ID
     * @param id Identifiant du produit à supprimer
     */
    public void deleteProduit(Long id);
    /**
     * Récupérer la liste de tous les produits
     * @return Liste des produits
     */
    public List<Produit> getAllProduits();
    /**
     * Récupérer un produit par son ID
     * @param id Identifiant du produit
     * @return Produit correspondant ou null si non trouvé
     */
    public Produit getProduitById(Long id);
    /**
     * Mettre à jour un produit existant
     * @param p Produit avec nouvelles informations
     */
    public void updateProduit(Produit p);

}
```

ProduitMetierImpl.java

```java
package services;

import java.util.List;

/*
 * Implémentation concrète de la couche métier (service layer).
 * Cette classe sert d'intermédiaire entre la couche Web (Servlets/JSP)
 * et la couche DAO (ProduitDAO). Elle encapsule la logique métier.
 *
 * Elle utilise le pattern Singleton pour s'assurer qu'il n'y ait qu'une
 * seule instance de la couche métier, ce qui permet de partager la même
 * liste de produits préchargée entre toutes les servlets.
 */

public class ProduitMetierImpl implements ProduitMetier {

    // Instance unique pour le singleton
    private static ProduitMetierImpl instance;

    // Référence vers la couche DAO
    private ProduitDAO dao;

    /*
     * Constructeur privé pour le singleton.
     * Initialise le DAO et précharge les produits via init().
     */
    private ProduitMetierImpl() {
        dao = new ProduitImpl();
        ((ProduitImpl) dao).init(); // Précharge des produits au démarrage
    }

    /*
     * Méthode pour récupérer l'instance unique du singleton.
     * Si elle n'existe pas, on la crée.
     */
    public static ProduitMetierImpl getInstance() {
        if (instance == null) {
            instance = new ProduitMetierImpl();
        }
        return instance;
    }

    /*
     * Ajouter un produit via le DAO.
     */
    @Override
    public void addProduit(Produit p) {
        dao.addProduit(p);
    }

    /*
     * Supprimer un produit via le DAO.
     */
    @Override
    public void deleteProduit(Long id) {
        dao.deleteProduit(id);
    }
    /*
     * Récupérer la liste complète des produits via le DAO.
     */
    @Override
    public List<Produit> getAllProduits() {
        return dao.getAllProduits();
    }

    /*
     * Récupérer un produit par son identifiant via le DAO.
     */
    @Override
    public Produit getProduitById(Long id) {
        return dao.getProduitById(id);
    }

    /*
     * Mettre à jour un produit existant via le DAO.
     */
    @Override
    public void updateProduit(Produit p) {
        dao.updateProduit(p);
    }
}
```

LoginMetier.java

```java
package services;

import dao.*;

public class LoginMetier {
    // Instance unique pour le singleton
    private static LoginMetier instance;

    // Référence vers la couche DAO
    private UserDAO userDao;

    /*
     * Constructeur privé pour le singleton.
     * Initialise le DAO et précharge les produits via init().
     */
    private LoginMetier() {
        userDao = new UserImpl();
        ((UserImpl) userDao).init(); // Précharge des produits au démarrage
    }

    /*
     * Méthode pour récupérer l'instance unique du singleton.
     * Si elle n'existe pas, on la crée.
     */
    public static LoginMetier getInstance() {
        if (instance == null) {
            instance = new LoginMetier();
        }
        return instance;
    }

    public boolean verifierIdentifiants(String email, String password) {
        User u = userDao.getUser(email, password);
        if (u != null)
            return true;
        return false;
    }

    public String getRole(String email, String password) {
        User u = userDao.getUser(email, password);
        return u.getRole();
    }
}
```

RegisterMetier.java

```java
package services;

import dao.*;

public class RegisterMetier {

    // Instance unique pour le singleton
    private static RegisterMetier instance;

    // Référence vers la couche DAO
    private UserDAO userDao;

    /*
     * Constructeur privé pour le singleton.
     * Initialise le DAO et précharge les produits via init().
     */
    private RegisterMetier() {
        userDao = new UserImpl();
        ((UserImpl) userDao).init(); // Précharge des produits au démarrage
    }

    /*
     * Méthode pour récupérer l'instance unique du singleton.
     * Si elle n'existe pas, on la crée.
     */
    public static RegisterMetier getInstance() {
        if (instance == null) {
            instance = new RegisterMetier();
        }
        return instance;
    }

    public boolean RegisterUser(User user) {
        User u = userDao.getUser(user.getEmail());
        if(u == null) {
            userDao.ajouterUser(user);
            return true;
        }
        return false;
    }
}
```

5. Couche Servlets (web)

ListProduitServlet.java

```java
package web;

import java.io.IOException;
import java.util.*;
import javax.servlet.ServletException;
import javax.servlet.http.*;
import dao.Produit;
import services.*;
/*
 * Servlet qui gère l'affichage de la liste des produits et supporte la recherche par ID.
 * Fonctionnalités :
 * 1. Si le paramètre idProduit est fourni et valide, afficher uniquement ce produit
 * 2. Sinon, afficher tous les produits
 * Partie "Controller" dans MVC :
 * JSP (formulaire de recherche ou page d'accueil) -> Servlet -> Couche métier -> DAO -> JSP
 */
public class ListProduitServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;
    // Singleton de la couche métier
    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        // Lecture du paramètre "idProduit" envoyé depuis le formulaire de recherche
        String idParam = req.getParameter("idProduit");
        // Liste de produits à afficher
        List<Produit> liste = new ArrayList<>();
        if (idParam != null && !idParam.isEmpty()) {
            try {
                // Conversion du paramètre en Long
                long id = Long.parseLong(idParam);
                // Recherche du produit correspondant via la couche métier
                Produit p = metier.getProduitById(id);
                if (p != null) {
                    liste.add(p); // ajouter le produit trouvé à la liste
                }
                // Si le produit n'existe pas, la liste restera vide (option : message d'erreur)
            } catch (NumberFormatException e) {
                // Cas où l'utilisateur entre un ID invalide (non numérique)
                // On récupère tous les produits pour ne pas bloquer la page
                liste = metier.getAllProduits();
            }
        } else {
            // Aucun paramètre fourni : on affiche tous les produits
            liste = metier.getAllProduits();
        }
        // Stocker la liste dans la requête pour l'affichage JSP
        req.setAttribute("listeProduits", liste);
        // Forward vers JSP pour affichage (on garde les attributs de la requête)
        req.getRequestDispatcher("/securise/home.jsp").forward(req, resp);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
        doGet(request, response);
    }
}
```

AddProduitServlet.java

```java
package web;

import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

import javax.servlet.ServletException;
import javax.servlet.http.*;

import dao.Produit;
import services.ProduitMetier;
import services.ProduitMetierImpl;

/*
 * Servlet responsable de l'ajout d'un produit.
 * Partie "Controller" de l'architecture MVC1.
 *
 * Rôle :
 * - Recevoir les données du formulaire (nom, description, prix)
 * - Créer un objet Produit
 * - Appeler la couche métier pour l'ajouter à la liste DAO
 * - Mettre à jour la liste complète des produits pour l'affichage
 * - Forwarder vers la JSP pour affichage
 */

public class AddProduitServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    // Instance singleton de la couche métier
    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    // Liste locale, utilisée seulement si on voulait gérer des produits temporaires (inutile ici)
    List<Produit> produits = new ArrayList<Produit>();

    /*
     * Méthode POST : reçoit le formulaire d'ajout d'un produit
     */
    protected void doPost(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {
        // Récupération des paramètres du formulaire
        String nom = req.getParameter("nom");
        String description = req.getParameter("description");
        Double prix = Double.parseDouble(req.getParameter("prix")); // conversion String -> Double

        // Création de l'objet Produit
        Produit p = new Produit(nom, description, prix);
        // Ajout via la couche métier (qui met à jour la liste DAO)
        metier.addProduit(p);
        // La liste locale produits.add(p) n'est pas nécessaire ici,
        // car metier.getAllProduits() contient déjà tous les produits.
        produits.add(p);
        // Mise à disposition de la liste complète pour la JSP
        req.setAttribute("listeProduits", metier.getAllProduits());
        // Forward vers la JSP (index.jsp) pour affichage
        // Important : on ne redirige pas, sinon la requête perd ses attributs
        req.getRequestDispatcher("/listProduits").forward(req, resp);
    }
}
```

DeleteProduitServlet.java

```java
package web;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.*;

import services.*;

/*
 * Servlet responsable de la suppression d'un produit.
 * Partie "Controller" dans MVC1.
 *
 * Flux :
 * 1. L'utilisateur clique sur "Supprimer" dans la JSP (index.jsp)
 * 2. Le lien contient l'ID du produit à supprimer
 * 3. La servlet récupère l'ID et appelle la couche métier
 * 4. La couche métier supprime le produit dans le DAO
 * 5. Redirection vers ListProduitServlet pour rafraîchir l'affichage
 */

public class DeleteProduitServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    // Singleton de la couche métier pour accéder à la liste des produits
    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    /*
     * Méthode GET utilisée pour recevoir l'ID du produit à supprimer.
     * On utilise GET ici car l'action est déclenchée par un lien <a href="deleteProduit?id=...">
     */
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp)
            throws ServletException, IOException {

        // Récupération du paramètre "id" depuis l'URL
        long id = Long.parseLong(req.getParameter("id"));

        // Appel à la couche métier pour supprimer le produit
        metier.deleteProduit(id);

        // Redirection vers ListProduitServlet pour afficher la liste mise à jour
        // Important : sendRedirect déclenche une nouvelle requête HTTP
        resp.sendRedirect(req.getContextPath() + "/listProduits");
    }

}
```

EditProduitServlet.java

```java
package web;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.*;

import dao.Produit;
import services.*;

/*
 * Servlet qui prépare un produit pour modification.
 * Rôle :
 * 1. Recevoir l'ID du produit à modifier depuis la JSP (via le lien "Modifier")
 * 2. Récupérer le produit correspondant via la couche métier
 * 3. Pré-remplir le formulaire de la JSP avec les données du produit
 * 4. Afficher la liste complète des produits également
 *
 * Flux MVC :
 * JSP (index.jsp) -> EditProduitServlet -> ProduitMetierImpl -> ProduitDAO -> JSP (index.jsp)
 */

public class EditProduitServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    // Singleton de la couche métier
    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        // Récupération de l'ID depuis le lien "Modifier"
        long id = Long.parseLong(req.getParameter("id"));

        // Récupération du produit correspondant via la couche métier
        Produit p = metier.getProduitById(id);

        // Ajout du produit dans les attributs de requête pour pré-remplir le formulaire
        req.setAttribute("produitEdit", p);

        // On ajoute aussi la liste complète pour afficher tous les produits
        req.setAttribute("listeProduits", metier.getAllProduits());

        // Forward vers la JSP index.jsp
        // Le formulaire sera pré-rempli si "produitEdit" existe
        req.getRequestDispatcher("/listProduits").forward(req, resp);

    }

}
```

UpdateProduitServlet.java

```java
package web;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.*;

import dao.Produit;
import services.*;

/*
 * Servlet responsable de la mise à jour d'un produit.
 * Flux MVC :
 * 1. L'utilisateur modifie les champs dans le formulaire pré-rempli (EditProduitServlet)
 * 2. Le formulaire envoie les données à UpdateProduitServlet via POST
 * 3. La servlet récupère les données, construit un objet Produit
 * 4. Appel de la couche métier pour mettre à jour le produit
 * 5. Forward vers JSP pour afficher la liste mise à jour
 */

public class UpdateProduitServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    // Singleton de la couche métier
    private static final ProduitMetier metier = ProduitMetierImpl.getInstance();

    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse rep) throws ServletException, IOException {
        doPost(req, rep);
    }

    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        // Récupération des paramètres du formulaire
        Long id = Long.parseLong(req.getParameter("idProduit"));
        String nom = req.getParameter("nom");
        String description = req.getParameter("description");
        Double prix = Double.parseDouble(req.getParameter("prix"));
        // Création d'un objet Produit avec les nouvelles valeurs
        Produit p = new Produit();
        p.setIdProduit(id);
        p.setNom(nom);
        p.setDescription(description);
        p.setPrix(prix);
        // Mise à jour via la couche métier
        metier.updateProduit(p);
        // Préparer la liste complète pour l'affichage JSP
        req.setAttribute("listeProduits", metier.getAllProduits());
        // Forward vers index.jsp pour afficher le tableau mis à jour
        // On utilise forward ici pour conserver les attributs de la requête
        req.getRequestDispatcher("/listProduits").forward(req, resp);
        // Remarque : si l'on utilisait sendRedirect("listProduits"),
        // on perdrait les attributs et il faudrait relire les produits via la servlet ListProduitServlet
    }

}
```

LoginServlet.java

```java
package web;

import java.io.IOException;
import javax.servlet.ServletException;
import javax.servlet.http.*;

//import dao.*;
import services.*;

public class LoginServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");

        String email = request.getParameter("email");
        String pass = request.getParameter("pass");

        final LoginMetier login = LoginMetier.getInstance();
        boolean estUtilisateur = login.verifierIdentifiants(email, pass);

        if (estUtilisateur) {
            // Creation de la session
            HttpSession session = request.getSession(true);
            session.setAttribute("userSession", login.getRole(email, pass));

            response.sendRedirect(request.getContextPath() + "/listProduits");
        } else {
            request.setAttribute("erreur", "Email ou mot de passe incorrect.");
            request.getRequestDispatcher("login.jsp").forward(request, response);
        }

    }

}
```

RegisterServlet.java

```java
package web;

import java.io.IOException;

public class RegisterServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        final RegisterMetier register = RegisterMetier.getInstance();

        String firstname = request.getParameter("firstname");
        String lastname = request.getParameter("lastname");
        String email = request.getParameter("email");
        String pass = request.getParameter("pass");

        if (firstname == null || lastname == null || email == null || pass == null || firstname.isEmpty()
                || lastname.isEmpty() || email.isEmpty() || pass.isEmpty()) {
            request.setAttribute("erreur", "Tous les champs sont obligatoires.");
            request.getRequestDispatcher("register.jsp").forward(request, response);
            return;
        }

        User nouvelUtilisateur = new User(firstname, lastname, email, pass, "client");

        if (register.RegisterUser(nouvelUtilisateur) == false) {
            request.setAttribute("erreur", "Cet email est déjà utilisé.");
            request.getRequestDispatcher("register.jsp").forward(request, response);
            return;
        }

        request.setAttribute("succes", "Inscription réussie ! Vous pouvez vous connecter.");
        request.getRequestDispatcher("login.jsp").forward(request, response);
    }

}
```

LogoutServlet.java

```java
package web;

import java.io.IOException;

public class LogoutServlet extends HttpServlet {
    private static final long serialVersionUID = 1L;

    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        HttpSession session = request.getSession(false);
        if (session != null) {
            session.invalidate(); // Détruit la session
        }
        response.sendRedirect(request.getContextPath() + "/login.jsp");
    }

}
```

6. Le composant de Sécurité

Filter.java

```java
package web;

import java.io.IOException;

@WebFilter("/securise/*")
public class Filter extends HttpFilter implements javax.servlet.Filter {
    private static final long serialVersionUID = 1L;

    public void init(FilterConfig fConfig) throws ServletException {}
    public void destroy() {}

    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
            throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        HttpServletResponse res = (HttpServletResponse) response;

        HttpSession session = req.getSession(false);

        boolean isConnecte = (session != null && session.getAttribute("userSession") != null);

        if (isConnecte) {
            chain.doFilter(request, response);
        } else {
            res.sendRedirect(req.getContextPath() + "/login.jsp");
        }
    }
}
```

7. Les Vues

index.jsp

```
1  <%@ page language="java" contentType="text/html; charset=UTF-8"
2      pageEncoding="UTF-8"%>
3  <!DOCTYPE html>
4  <html>
5  <head>
6  <meta charset="UTF-8">
7  <title>Home</title>
8  </head>
9  <body>
10     <a href="login.jsp" class="btn btn-green">Login</a><br>
11     <a href="register.jsp" class="btn btn-blue">Register</a>
12 </body>
13 </html>
```

login.jsp

```
1  <%@ page language="java" contentType="text/html; charset=UTF-8"
2      pageEncoding="UTF-8"%>
3  <!DOCTYPE html>
4  <html>
5  <head>
6  <meta charset="UTF-8">
7  <title>Connexion</title>
8  </head>
9  <body>
10     <h2>Connexion</h2><br>
11
12     <div style="color: red;">${erreur}</div>
13     <div style="color: green;">${succes}</div><br>
14
15     <form action="LoginServlet" method="post">
16         <label for="email">Email :</label>
17         <input type="email" name="email" id="email"><br>
18         <label for="pass">Password :</label>
19         <input type="password" name="pass" id="pass"><br>
20         <input type="submit" value="Se connecter">
21     </form>
22     <br>
23     <a href="register.jsp">Pas encore de compte? Inscrivez-vous</a>
24 </body>
25 </html>
```

register.java

```
1  <%@ page language="java" contentType="text/html; charset=UTF-8"
2      pageEncoding="UTF-8"%>
3  <!DOCTYPE html>
4  <html>
5  <head>
6  <meta charset="UTF-8">
7  <title>Inscription</title>
8  </head>
9  <body>
10     <h2>Créer un compte</h2>
11
12     <div style="color: red;">${erreur}</div>
13
14     <form action="RegisterServlet" method="post">
15         <label for="lastname">Nom :</label>
16         <input type="text" name="lastname" id="lastname"><br>
17         <label for="firstname">Prénom :</label>
18         <input type="text" name="firstname" id="firstname"><br>
19         <label for="email">Email :</label>
20         <input type="email" name="email" id="email"><br>
21         <label for="pass">Password :</label>
22         <input type="password" name="pass" id="pass"><br>
23         <input type="submit" value="S'inscrire">
24     </form>
25     <br>
26     <a href="login.jsp">Déjà inscrit? Connectez-vous</a>
27 </body>
28 </html>
```

erreur.java

```
1  <%@ page language="java" contentType="text/html; charset=UTF-8"
2      pageEncoding="UTF-8" isErrorPage="true"%>
3  <!DOCTYPE html>
4  <html>
5  <head>
6  <title>Oups ! Une erreur est survenue</title>
7  </head>
8  <body>
9      <div
10         style="border: 2px solid red; padding: 20px; max-width: 600px; margin: 0 auto; text-align: center;">
11         <h1 style="color: red;">Désolé, un problème est survenu.</h1>
12
13         <!-- Récupération du code d'erreur HTTP (ex: 404, 500) -->
14         <h2>
15             Code d'erreur :
16             <%=request.getAttribute("javax.servlet.error.status_code")%></h2>
17
18         <p>
19             <strong>Description :</strong>
20             <!-- Récupération du message d'erreur HTTP -->
21             <%=request.getAttribute("javax.servlet.error.message")%>
22         </p>
23
24         <!-- Affichage de l'exception Java s'il y en a une (ex: NullPointerException) -->
25         <%
26         if (exception != null) {
27         %>
28         <p>
29             <strong>Détail technique :</strong>
30             <%=exception.getMessage()%></p>
31         <%
32         }
33         %>
34
35         <br> <a href="${pageContext.request.contextPath}/login.jsp">Retourner
36             à la page de se connecter</a>
37     </div>
38 </body>
39 </html>
```

home.jsp

```jsp
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core"%>

<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Gestion des Produits MVC1</title>
</head>
<body>
    <h2>Gestion des Produits (MVC1)</h2>

    <!-- Récupération le nom de l'utilisateur depuis la session -->
    <p>Role: <strong>${sessionScope.userSession}</strong></p>
    <!--
        Formulaire Ajouter / Modifier
        - Utilise le même formulaire pour créer ou modifier un produit
        - Si un produit est chargé pour modification (produitEdit != null)
        => action pointe vers UpdateProduitServlet
        - Sinon => action pointe vers AddProduitServlet
    -->

    <c:if test="${sessionScope.userSession == 'admin'}">
        <form action="${pageContext.request.contextPath}/${produitEdit != null ? 'updateProduit' : 'addProduit'}"
            method="post">
            <!-- Champ caché pour l'ID du produit à modifier -->
            <input type="hidden" name="idProduit" value="${produitEdit.idProduit}" />
            Nom: <input type="text" name="nom" value="${produitEdit.nom}" required />
            Description: <input type="text" name="description"
                value="${produitEdit.description}" required /> Prix: <input
                type="text" name="prix" value="${produitEdit.prix}" required />
            <!-- Champ caché pour l'ID du produit à modifier -->
            <input type="submit"
                value="${produitEdit != null ? 'Modifier' : 'Ajouter'}" />
        </form>
    </c:if>

    <hr/>

    <!--
        Formulaire Recherche
        - Redirige vers ListProduitServlet
        - Si un ID est fourni, seul ce produit est affiché
        - Sinon, tous les produits sont listés
    -->
    <form action="${pageContext.request.contextPath}/listProduits" method="get">
        ID: <input type="text" name="idProduit" /> <input type="submit"
            value="Rechercher" />
    </form>
    <hr />

    <!--
        Tableau affichant la liste des produits
        - Utilise JSTL cForEach pour parcourir listeProduits
        - Les actions "Modifier" et "Supprimer" appellent respectivement
          EditProduitServlet et DeleteProduitServlet
    -->
   <table border="1" cellpadding="5">
       <tr>
           <th>ID</th>
           <th>Nom</th>
           <th>Description</th>
           <th>Prix</th>
           <th>Actions</th>
       </tr>
       <c:forEach var="p" items="${listeProduits}">
           <tr>
               <td>${p.idProduit}</td>
               <td>${p.nom}</td>
               <td>${p.description}</td>
               <td>${p.prix}</td>

               <c:if test="${sessionScope.userSession == 'admin'}">
                   <td>
                       <!-- Modifier : charge le produit dans le formulaire pour édition -->
                       <a href="editProduit?id=${p.idProduit}">Modifier</a> | <!-- Modifier : charge le produit dans le formulaire pour édition -->
                       <a href="deleteProduit?id=${p.idProduit}"
                       onclick="return confirm('Voulez-vous vraiment supprimer ce produit ?');">Supprimer</a>
                   </td>
               </c:if>
           </tr>
       </c:forEach>
   </table>

   <br>
   <form action="${pageContext.request.contextPath}/LogoutServlet">
       <input type="submit" value="Se déconnecter">
   </form>

</body>
</html>
```